Candidate Name:	Richard John	**Reg / Matric No:**	**2024/C/DSC/0548**
Department:	Data Science	**Institutional Email:**	richard.john@miva.edu.ng

Source Code & Verification Repository: https://github.com/Rik-ky/miva-csc301-data-structures

Question 1: Implementing a Custom Singly Linked List

Objective & Implementation Scope: A robust, object-oriented C++ program was designed to implement a custom singly linked list from first principles. The implementation features modular methods to add nodes at the beginning, middle (by 1-based index), and end of the list, delete nodes by value or by position, display all nodes, and enforce comprehensive error handling for edge conditions including deletions on empty lists and out-of-bounds positions.

Memory Management Analysis (100–150 words):

In a C++ singly linked list, memory is dynamically partitioned between the heap and the call stack. When an insertion is invoked, the new operator dynamically allocates a Node structure on the free store (heap), reserving memory for its integer data payload and a pointer to the next node (Node* next). The stack maintains only lightweight references, primarily the head pointer and transient traversal variables. Unlike contiguous arrays, linked list nodes reside at non-contiguous heap addresses, interconnected solely by forward pointers. During deletion, target nodes are unlinked by updating the preceding node's next pointer, and their heap memory is explicitly freed using delete to prevent memory leaks and dangling pointers. Finally, the class destructor iterates through all remaining nodes, deallocating each sequentially in $O(n)$ time to ensure complete resource reclamation.

© Copyright 2024 MIVA Open University All Rights Reserved

Question 1: C++ Implementation (Node Struct & Insertions)

The listing below presents the node structure and insertion routines at the beginning, end, and middle of the linked list:

```
// ================================================================================
// Question 1: Custom Singly Linked List (Node Definition & Insertions)
// Course: CSC 301 - Data Structures | Miva Open University
// Candidate: Richard John (2024/C/DSC/0548) | richard.john@miva.edu.ng
// ================================================================================
#include <iostream>

struct Node {
    int data;       // Node integer payload
    Node* next;     // Forward pointer to subsequent node
    Node(int value) : data(value), next(nullptr) {}
};

class SinglyLinkedList {
private:
    Node* head;     // Pointer to the first node
    int size;       // Number of active nodes

public:
    SinglyLinkedList() : head(nullptr), size(0) {}
    ~SinglyLinkedList() { clear(); }

    bool isEmpty() const { return head == nullptr; }
    int getSize() const { return size; }

    // 1. Add a node to the beginning of the list
    void insertAtBeginning(int value) {
        Node* newNode = new Node(value);
        newNode->next = head;
        head = newNode;
        size++;
        std::cout << "[SUCCESS] Inserted " << value << " at the beginning.\n";
    }

    // 2. Add a node to the end of the list
    void insertAtEnd(int value) {
        Node* newNode = new Node(value);
        if (isEmpty()) {
            head = newNode;
        } else {
            Node* current = head;
            while (current->next != nullptr) current = current->next;
            current->next = newNode;
        }
        size++;
        std::cout << "[SUCCESS] Inserted " << value << " at the end.\n";
    }

    // 3. Add a node at a specific position (1-based index)
    void insertAtPosition(int value, int position) {
        if (position < 1 || position > size + 1) {
            std::cout << "[ERROR] Invalid position " << position
                      << ". Valid range is 1 to " << (size + 1) << ".\n";
            return;
        }
        if (position == 1) {
            insertAtBeginning(value);
            return;
        }
        Node* newNode = new Node(value);
        Node* current = head;
        for (int i = 1; i < position - 1; ++i) current = current->next;
        newNode->next = current->next;
        current->next = newNode;
        size++;
        std::cout << "[SUCCESS] Inserted " << value << " at position " << position << ".\n";
    }
```

© Copyright 2024 MIVA Open University All Rights Reserved

Question 1: C++ Implementation (Deletions & Traversal)

The listing below implements node deletions by value and position, defensive error handling, traversal display, and heap deallocation:

```
    // 4. Delete a node by value with defensive error handling
    bool deleteByValue(int value) {
        if (isEmpty()) {
            std::cout << "[ERROR] Cannot delete from an empty list.\n";
            return false;
        }
        if (head->data == value) {
            Node* temp = head; head = head->next; delete temp; size--;
            std::cout << "[SUCCESS] Deleted node with value " << value << " (head node).\n";
            return true;
        }
        Node* current = head;
        while (current->next != nullptr && current->next->data != value) current = current->next;
        if (current->next == nullptr) {
            std::cout << "[ERROR] Value " << value << " not found in the list.\n";
            return false;
        }
        Node* nodeToDelete = current->next;
        current->next = nodeToDelete->next;
        delete nodeToDelete;
        size--;
        std::cout << "[SUCCESS] Deleted node with value " << value << ".\n";
        return true;
    }

    // 5. Delete a node by position (1-based index)
    bool deleteByPosition(int position) {
        if (isEmpty()) {
            std::cout << "[ERROR] Cannot delete from an empty list.\n";
            return false;
        }
        if (position < 1 || position > size) {
            std::cout << "[ERROR] Invalid position " << position << ". Valid range is 1 to " << size << ".\n";
            return false;
        }
        if (position == 1) {
            Node* temp = head; head = head->next; int val = temp->data; delete temp; size--;
            std::cout << "[SUCCESS] Deleted node at position 1 (value: " << val << ").\n";
            return true;
        }
        Node* current = head;
        for (int i = 1; i < position - 1; ++i) current = current->next;
        Node* nodeToDelete = current->next;
        int val = nodeToDelete->data;
        current->next = nodeToDelete->next;
        delete nodeToDelete;
        size--;
        std::cout << "[SUCCESS] Deleted node at position " << position << " (value: " << val << ").\n";
        return true;
    }

    // 6. Display all nodes in the list
    void display() const {
        if (isEmpty()) { std::cout << "List: [ EMPTY ] (Size: 0)\n"; return; }
        std::cout << "List: [ ";
        Node* current = head;
        while (current != nullptr) {
            std::cout << current->data;
            if (current->next != nullptr) std::cout << " -> ";
            current = current->next;
        }
        std::cout << " -> NULL ] (Size: " << size << ")\n";
    }

    // Deallocate all nodes on the heap
    void clear() {
        Node* current = head;
        while (current != nullptr) {
            Node* temp = current; current = current->next; delete temp;
        }
        head = nullptr; size = 0;
    }
};
```

© Copyright 2024 MIVA Open University All Rights Reserved

Question 1 Verified Execution Outputs

The program was tested with three distinct datasets: (1) Standard operations, (2) Edge cases and error handling, and (3) Practical student ID batch processing:

```
======================================================================
 MIVA OPEN UNIVERSITY | CSC 301: DATA STRUCTURES TMA
 Candidate: Richard John (Matric: 2024/C/DSC/0548)
 Question 1: Custom Singly Linked List Verification
======================================================================

>>> TEST SET 1: Standard Insertions and Deletions
----------------------------------------------------------------------
List: [ EMPTY ] (Size: 0)
[SUCCESS] Inserted 30 at the beginning.
[SUCCESS] Inserted 10 at the beginning.
[SUCCESS] Inserted 50 at the end.
List: [ 10 -> 30 -> 50 -> NULL ] (Size: 3)
[SUCCESS] Inserted 20 at position 2.
List: [ 10 -> 20 -> 30 -> 50 -> NULL ] (Size: 4)
[SUCCESS] Inserted 40 at position 4.
List: [ 10 -> 20 -> 30 -> 40 -> 50 -> NULL ] (Size: 5)

-- Performing Deletions on Test Set 1 --
[SUCCESS] Deleted node with value 30.
List: [ 10 -> 20 -> 40 -> 50 -> NULL ] (Size: 4)
[SUCCESS] Deleted node at position 1 (value: 10).
List: [ 20 -> 40 -> 50 -> NULL ] (Size: 3)
[SUCCESS] Deleted node at position 3 (value: 50).
List: [ 20 -> 40 -> NULL ] (Size: 2)

>>> TEST SET 2: Edge Cases and Defensive Error Handling
----------------------------------------------------------------------
[INFO] List cleared for edge case validation.
List: [ EMPTY ] (Size: 0)

Attempting deletion on empty list:
[ERROR] Cannot delete from an empty list.
[ERROR] Cannot delete from an empty list.

Attempting out-of-bounds insertions:
[ERROR] Invalid position 0. Valid range is 1 to 1.
[ERROR] Invalid position 5. Valid range is 1 to 1.

Operating on a single-element list:
[SUCCESS] Inserted 77 at the beginning.
List: [ 77 -> NULL ] (Size: 1)
[ERROR] Value 999 not found in the list.
[SUCCESS] Deleted node at position 1 (value: 77).
List: [ EMPTY ] (Size: 0)

>>> TEST SET 3: Practical Batch Processing (Student ID Records)
----------------------------------------------------------------------
[SUCCESS] Inserted 101 at the end.
[SUCCESS] Inserted 102 at the end.
[SUCCESS] Inserted 104 at the end.
[SUCCESS] Inserted 105 at the end.
List: [ 101 -> 102 -> 104 -> 105 -> NULL ] (Size: 4)

Inserting missing student ID 103 at position 3:
[SUCCESS] Inserted 103 at position 3.
List: [ 101 -> 102 -> 103 -> 104 -> 105 -> NULL ] (Size: 5)

Inserting transfer student ID 100 at beginning:
[SUCCESS] Inserted 100 at the beginning.
List: [ 100 -> 101 -> 102 -> 103 -> 104 -> 105 -> NULL ] (Size: 6)

Removing withdrawn student ID 104 by value:
[SUCCESS] Deleted node with value 104.
List: [ 100 -> 101 -> 102 -> 103 -> 105 -> NULL ] (Size: 5)

Clearing all remaining records:
List: [ EMPTY ] (Size: 0)

[ALL TESTS COMPLETED SUCCESSFULLY]
```

© Copyright 2024 MIVA Open University All Rights Reserved

Question 2: Solving Problems Using Recursion

Objective & Recursive Implementations: A modular C++ program was implemented to solve four classic algorithmic problems using recursion: (1) Factorial calculation, (2) Fibonacci sequence generation, (3) In-place/substring String Reversal, and (4) Recursive Binary Search on sorted arrays. Each function defines rigorous base and recursive cases.

Base Case and Recursive Case Analysis:

Function	Base Case	Recursive Case	Time / Space Complexity
Factorial	n <= 1: returns 1	n * factorial(n - 1)	Time: $O(n)$ Space: $O(n)$ stack
Fibonacci	n <= 0: 0 n == 1: 1	fib(n - 1) + fib(n - 2)	Time: $O(2^n)$ Space: $O(n)$ stack
String Reversal	str.length() <= 1: return str	reverse(str.substr(1)) + str[0]	Time: $O(n^2)$ (substr) Space: $O(n)$ stack
Binary Search	low > high: -1 arr[mid] == key: mid	key < arr[mid]: left key > arr[mid]: right	Time: $O(\log n)$ Space: $O(\log n)$ stack

Recursion vs. Iteration Comparative Summary (150–200 words):

Recursion and iteration are fundamental computational approaches for repetitive problem solving. In recursion, a function calls itself to resolve progressively smaller sub-problems until reaching a terminating base case. Each recursive invocation pushes an activation record onto the call stack, allocating memory for parameters, local variables, and return addresses. Consequently, recursion incurs $O(n)$ auxiliary stack space, runtime function-call overhead, and vulnerability to stack overflow if recursion depth is unbounded. Conversely, iteration relies on looping constructs managed by loop variables residing in constant stack space ($O(1)$ memory), providing superior execution speed and cache locality. However, recursion is strongly preferred in problems with self-similar, branching, or divide-and-conquer structures where iterative equivalents require complex manual stack management. Primary scenarios favoring recursion include hierarchical data structures (binary tree traversals, trie operations), graph depth-first search (DFS), divide-and-conquer paradigms (Merge Sort, Quick Sort, Binary Search), and backtracking algorithms (N-Queens, maze traversal), where recursive formulations deliver unmatched mathematical clarity and maintainability.

© Copyright 2024 MIVA Open University All Rights Reserved

Question 2: C++ Implementation (Factorial, Fibonacci, String)

The listing below implements the recursive functions for factorial calculation, Fibonacci sequence generation, and string reversal:

```
// ================================================================================
// Question 2: Solving Problems Using Recursion (Part 1)
// Course: CSC 301 - Data Structures | Miva Open University
// Candidate: Richard John (2024/C/DSC/0548) | richard.john@miva.edu.ng
// ================================================================================
#include <iostream>
#include <string>

// 1. FACTORIAL CALCULATION
// Base Case: n <= 1 returns 1 (handles 0! = 1, 1! = 1)
// Recursive Case: n * factorial(n - 1)
long long factorial(int n) {
    if (n < 0) {
        std::cout << "[ERROR] Factorial is undefined for negative integers (" << n << "). ";
        return -1;
    }
    if (n == 0 || n == 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

// 2. FIBONACCI SEQUENCE
// Base Case: n <= 0 returns 0; n == 1 returns 1
// Recursive Case: fibonacci(n - 1) + fibonacci(n - 2)
int fibonacci(int n) {
    if (n <= 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

void printFirstNFibonacci(int n) {
    if (n <= 0) {
        std::cout << "[ERROR] Number of terms must be greater than 0.\n";
        return;
    }
    std::cout << "First " << n << " terms: ";
    for (int i = 0; i < n; ++i) {
        std::cout << fibonacci(i);
        if (i < n - 1) std::cout << ", ";
    }
    std::cout << "\n";
}

// 3. STRING REVERSAL
// Base Case: String length <= 1 returns string as is
// Recursive Case: reverse(rest of string) + first character
std::string reverseString(const std::string& str) {
    if (str.length() <= 1) {
        return str;
    }
    return reverseString(str.substr(1)) + str[0];
}
```

© Copyright 2024 MIVA Open University All Rights Reserved

Question 2: C++ Implementation (Binary Search & Runner)

The listing below implements recursive binary search with mid-overflow prevention and the multi-input test harness:

```
// 4. RECURSIVE BINARY SEARCH
// Base Cases: low > high (not found: -1) | arr[mid] == target (found: mid)
// Recursive Cases: target < arr[mid] (search left) | target > arr[mid] (search right)
int binarySearchRecursive(const int arr[], int low, int high, int target) {
    if (low > high) {
        return -1; // Base case: element not present
    }
    int mid = low + (high - low) / 2; // Prevents overflow
    if (arr[mid] == target) {
        return mid; // Base case: target located
    }
    if (target < arr[mid]) {
        return binarySearchRecursive(arr, low, mid - 1, target);
    }
    return binarySearchRecursive(arr, mid + 1, high, target);
}

int main() {
    // Part A: Factorial Tests
    std::cout << ">>> 1. FACTORIAL TESTS\n";
    std::cout << "Test 1 [0!]:        " << factorial(0) << " (Expected: 1)\n";
    std::cout << "Test 2 [5!]:        " << factorial(5) << " (Expected: 120)\n";
    std::cout << "Test 3 [10!]:       " << factorial(10) << " (Expected: 3628800)\n";
    std::cout << "Test 4 [-4!]:       " << factorial(-4) << " (Handled)\n\n";

    // Part B: Fibonacci Tests
    std::cout << ">>> 2. FIBONACCI TESTS\n";
    std::cout << "Test 1 [n=1]:  "; printFirstNFibonacci(1);
    std::cout << "Test 2 [n=7]:  "; printFirstNFibonacci(7);
    std::cout << "Test 3 [n=12]: "; printFirstNFibonacci(12);
    std::cout << "Test 4 [n=0]:  "; printFirstNFibonacci(0);
    std::cout << "\n";

    // Part C: String Reversal Tests
    std::cout << ">>> 3. STRING REVERSAL TESTS\n";
    std::cout << "Test 1 [\"\"]:       \"" << reverseString("") << "\"\n";
    std::cout << "Test 2 [\"racecar\"]:  \"" << reverseString("racecar") << "\"\n";
    std::cout << "Test 3 [\"Data Structures\"]: \"" << reverseString("Data Structures") << "\"\n\n";

    // Part D: Binary Search Tests
    std::cout << ">>> 4. BINARY SEARCH TESTS\n";
    int sortedArray[] = {4, 11, 19, 28, 35, 42, 58, 67, 73, 89, 95};
    int n = sizeof(sortedArray) / sizeof(sortedArray[0]);
    std::cout << "Test 1 [Find 42]:  Index " << binarySearchRecursive(sortedArray, 0, n - 1, 42) << "\n";
    std::cout << "Test 2 [Find 4]:   Index " << binarySearchRecursive(sortedArray, 0, n - 1, 4) << "\n";
    std::cout << "Test 3 [Find 50]:  Result " << binarySearchRecursive(sortedArray, 0, n - 1, 50) << " (Not found)\n";
    int emptyArray[] = {};
    std::cout << "Test 4 [Empty]:    Result " << binarySearchRecursive(emptyArray, 0, -1, 10) << " (Safe)\n";
    return 0;
}
```

© Copyright 2024 MIVA Open University All Rights Reserved

Question 2 Verified Execution Outputs

The terminal capture below validates all four recursive problems across standard inputs and boundary edge cases (0!, negative inputs, empty strings, and empty array binary searches):

```
======================================================================
 MIVA OPEN UNIVERSITY | CSC 301: DATA STRUCTURES TMA
 Candidate: Richard John (Matric: 2024/C/DSC/0548)
 Question 2: Solving Problems Using Recursion
======================================================================

>>> 1. FACTORIAL TESTS (Base Case: n <= 1 -> 1 | Recursive: n * f(n-1))
----------------------------------------------------------------------
Test 1 [Edge Case: 0!]:        1 (Expected: 1)
Test 2 [Standard: 5!]:         120 (Expected: 120)
Test 3 [Large Value: 10!]:     3628800 (Expected: 3628800)
Test 4 [Negative Input: -4!]:  [ERROR] Factorial is undefined for negative integers (-4). -1 (Error Handled)

>>> 2. FIBONACCI SEQUENCE TESTS (Base: n<=0->0, n=1->1 | Rec: f(n-1)+f(n-2))
----------------------------------------------------------------------
Test 1 [Single term (n=1)]:    First 1 terms: 0
Test 2 [Standard (n=7)]:       First 7 terms: 0, 1, 1, 2, 3, 5, 8
Test 3 [Extended (n=12)]:      First 12 terms: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89
Test 4 [Invalid (n=0)]:        [ERROR] Number of terms must be greater than 0.

>>> 3. STRING REVERSAL TESTS (Base: len<=1 -> str | Rec: rev(sub)+first)
----------------------------------------------------------------------
Test 1 [Empty String ""]:       "" (Length: 0)
Test 2 [Palindrome "racecar"]:  "racecar"
Test 3 [Phrase "Data Structures"]: "serutcurtS ataD"
Test 4 [Phrase "Miva Open University"]: "ytisrevinU nepO aviM"

>>> 4. RECURSIVE BINARY SEARCH TESTS
----------------------------------------------------------------------
Sorted Array: [ 4, 11, 19, 28, 35, 42, 58, 67, 73, 89, 95 ]
Test 1 [Find 42]:  Found at index 5 (Value: 42)
Test 2a [Find first 4]: Found at index 0
Test 2b [Find last 95]:  Found at index 10
Test 3 [Search non-existent 50]: Result = -1 (Correctly reported as NOT FOUND)
Test 4 [Search in Empty Array]:    Result = -1 (Handled safely without segmentation fault)

======================================================================
 [ALL RECURSION TESTS COMPLETED SUCCESSFULLY]
======================================================================
```

Submission Conclusion: All requirements for CSC 301 have been implemented with verified numerical and logical precision. Complete source code and output logs are published in the GitHub repository: https://github.com/Rik-ky/miva-csc301-data-structures.

© Copyright 2024 MIVA Open University All Rights Reserved